Operating Systems

ASSIGNMENT 1

NAME:j.manoj

REG NO:192572143

SERIAL NO:18

CO1: *Apply the concepts of operating system types, structures and system calls to demonstrate their functions in various computing environments – history of operating systems, mainframe, desktop, multiprocessor, distributed, clustered, real-time and handheld systems, OS structure, services, system calls, system design, CPU states, memory hierarchy and I/O systems.*

Assignment Title: Evaluate and Justify the Operating-System Design of a Smart Hospital (Total Marks - 50)

Nature of Assignment: Individual
Duration: 1 Week
Submission Date: 03/11/2026

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment1** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: https://github.com/navyareddy35/CSA0403/tree/main/Assignment-1

How to Answer the Additional Question

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Operating Systems for a Smart Hospital

A 500-bed hospital is replacing its computer systems. It needs: bedside patient monitors that must raise an alarm within 10 milliseconds of an abnormal reading; nurses' handheld tablets; doctors' desktop workstations; a central database server for patient records that must never stop; an imaging server that processes MRI scans using many processors; and a link to two partner hospitals that share records.

The IT team has proposed using one general-purpose operating system everywhere. The clinical staff are concerned that alarms may be delayed, the database may stop during updates, and the tablets' batteries will not last a shift.

The number of devices may grow:

1 ward (40 monitors, 10 tablets) → 1 hospital (500 monitors, 120 tablets, 80 workstations) → a group of 3 hospitals sharing records

You are required to evaluate the requirements of every part of the system, judge which type and structure of operating system suits each, examine how programs obtain OS services through system calls, and defend a complete operating-system plan for the hospital.

Design Challenge:

The hospital is adding remote patient monitoring at home and a cloud-based records service.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Evaluate the Requirements of Each Computing Environment 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate what each part of the hospital needs from its operating system.

Discuss:

- Response-time, reliability, throughput, battery and security requirements of each device
- Which requirements conflict (for example: fast response and many users on one machine)
- Which requirements are critical to patient safety
- How the lessons from the history of operating systems (batch, time-sharing, personal, mobile) apply to each environment

You are expected to **rank the requirements of each environment and justify the ranking**.

Alternative Concept and Complexity Analysis

If the whole hospital used a single general-purpose operating system on every device, instead of a different type of OS for each environment, what would the solution be? Evaluate the worst-case alarm response time (unbounded scheduling delay versus a guaranteed deadline), the memory needed on each device, and the administration effort.

Evidence Expected

A ranked requirement table for every environment with a justification for each ranking.

TASK 2 – Judge the Type of Operating System for Each Environment **3 Marks**

Bloom's Level: K5 – Evaluate

Compare the types of operating systems for the hospital.

Discuss:

- Mainframe, desktop, multiprocessor, distributed, clustered, real-time (hard and soft) and handheld systems
- Which type best fits the monitors, tablets, workstations, database server, imaging server and partner link
- Symmetric versus asymmetric multiprocessing for the imaging server
- A clustered system with fail-over for the database server

Alternative Concept and Complexity Analysis

If the database server used a single large multiprocessor machine instead of a cluster of smaller machines with fail-over, what would the solution be? Evaluate the availability (a single point of failure versus fail-over within seconds), the cost, and how throughput grows with the number of processors (time) and the memory available (space).

Evidence Expected

A justified choice of operating-system type for each environment, with the rejected alternatives and reasons.

TASK 3 – Evaluate Operating-System Structures **4 Marks**

Bloom's Level: K5 – Evaluate

Evaluate the possible structures of the operating system.

Discuss:

- Simple (monolithic), layered, microkernel, modular (loadable kernel modules) and hybrid structures
- The effect of each structure on reliability, performance and ease of updating
- Which structure suits the patient monitors and which suits the database server
- How Linux, Windows, Android and a real-time OS such as FreeRTOS or QNX are structured

Alternative Concept and Complexity Analysis

If the bedside monitors used a monolithic kernel instead of a microkernel, what would the solution be? Evaluate the time cost of a service request (one system call versus message passing between servers), the memory taken by the kernel, and the effect of a faulty driver on the whole system.

Evidence Expected

A comparison of structures against the hospital's requirements, with a defended choice for two environments.

TASK 4 – Evaluate the Services an Operating System Must Provide **4 Marks**

Bloom's Level: K5 – Evaluate

Evaluate the services each environment needs.

Discuss:

- User interface, program execution, I/O operations, file systems, communication, error detection, resource allocation, accounting, protection and security
- Which services can be removed from the bedside monitor to make it smaller and faster
- Which services are essential for the database server
- How the OS protects one patient's records from another user's programs

Alternative Concept and Complexity Analysis

If the tablets used a full desktop operating system with every service enabled instead of a reduced handheld OS, what would the solution be? Evaluate the memory and storage used by the extra services (space), the start-up time and the battery life (time).

Evidence Expected

A service table for each environment with justified inclusions and exclusions.

TASK 5 – Examine System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Examine how a program obtains OS services.

Discuss:

- The categories of system calls (process control, file management, device management, information maintenance, communication, protection)
- What happens step by step when a program calls read(): library call, trap, mode switch, kernel work, return
- How parameters are passed to the kernel (registers, a block in memory, the stack)
- How strace shows the system calls made by a program

Alternative Concept and Complexity Analysis

If programs called the C library functions fopen, fread and fwrite instead of the open, read and write system calls directly, what would the solution be? Evaluate the number of system calls for a file of n bytes (about n / 4096 with library buffering versus n / b for a buffer of b bytes) and the extra buffer space used by the library.

Evidence Expected

A step-by-step explanation of a system call with strace evidence from a real program.

TASK 6 – Evaluate the Cost of System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how the number of system calls affects performance.

Discuss:

- Why each system call costs much more than an ordinary function call
- How the buffer size changes the number of read() and write() calls when the database copies a file

- How library buffering (fread, fwrite) reduces the number of system calls
- The best buffer size for the hospital's backup program, using your measurements

Complete the Code

Complete TODO 1 below using the read() and write() system calls, and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

/* Copy src to dst with the read() and write() system calls, using a buffer of
   bufsize bytes. Returns the number of read() calls made, or -1 on error. */
long copy_file(const char *src, const char *dst, size_t bufsize)
{
    char buf[8192];
    int in = open(src, O_RDONLY);
    if (in < 0) return -1;
    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) { close(in); return -1; }
    long calls = 0;
    ssize_t n;
    // TODO 1: Loop: read up to bufsize bytes into buf (count every read call in calls);
    // stop when read returns 0 (end of file) or -1; write exactly the n bytes that
    // were read

    close(in);
    close(out);
    return calls;
}
int main(void)
{
    FILE *f = fopen("records.dat", "w");     /* a 10,000-byte test file */
    for (int i = 0; i < 10000; i++) fputc('A' + i % 26, f);
    fclose(f);

    size_t sizes[] = {1, 512, 4096};
    for (int i = 0; i < 3; i++)
        printf("buffer %4zu bytes: %5ld read() calls\n", sizes[i],
               copy_file("records.dat", "copy.dat", sizes[i]));
    unlink("records.dat");
    unlink("copy.dat");
    return 0;
}
```

Expected output:

```
buffer1 bytes: 10001 read() calls
buffer 512 bytes:    21 read() calls
buffer 4096 bytes:   4 read() calls
```

Run the program under strace -c and compare the number of read() calls reported with the program's output.

Alternative Concept and Complexity Analysis

If the backup program used memory-mapped files (mmap) instead of read() and write() with a buffer, what would the solution be? Evaluate the number of system calls (a few versus about n / b), the copying between kernel and user space, and the memory used for very large files.

Evidence Expected

Measured system-call counts and times for different buffer sizes, with a justified recommendation.

TASK 7 – Evaluate Process Control System Calls **4 Marks**

Bloom's Level: K5 – Evaluate

Evaluate how a command shell on the database server starts and manages programs.

Discuss:

- What fork(), exec(), wait() and exit() each do
- How a shell runs a command, and how it obtains the command's exit status
- What a zombie and an orphan process are, and how the OS deals with them
- Why fork() followed by exec() is used instead of a single "create process" call

Complete the Code

Complete TODO 1 below using fork(), execlp() and waitpid(), and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    printf("shell: running job 1\n");
    fflush(stdout);                          /* empty the buffer before fork() */
    // TODO 1: create a child with fork(). In the child, replace the program with
    //      execlp("expr", "expr", "6", "*", "7", (char *)NULL) and exit(127) if exec
    //      fails. In the parent, wait for the child with waitpid() and print "job 1
    exit
    //      status N" using WEXITSTATUS

    printf("shell: running job 2\n");
    fflush(stdout);
    pid = fork();
    if (pid == 0) _exit(3);                  /* a child that fails with code 3 */
    waitpid(pid, &status, 0);
    printf("job 2 exit status %d\n",
    WEXITSTATUS(status)); return 0;
}
```

Expected output:

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

Explain why stdout is flushed before each fork(). Then remove the second fflush(stdout), change _exit(3) to exit(3), redirect the output to a file and explain why "shell: running job 2" now appears twice.

Alternative Concept and Complexity Analysis

If commands were run with posix_spawn() or with threads instead of fork() followed by exec(), what would the solution be? Evaluate the time to start 1,000 commands, the memory copied or shared (copy-on-write), and the isolation between commands.

Evidence Expected

A working program that runs commands like a shell, with process diagrams and a reasoned evaluation.

TASK 8 – Evaluate CPU States and Protection — 3 Marks

Bloom's Level: K5 – Evaluate

Evaluate how the hardware protects the hospital system.

Discuss:

- User mode and kernel mode, privileged instructions and the mode bit
- Interrupts, traps and the timer interrupt
- What would happen to the monitors if any program could disable interrupts
- How the hardware stops one program from reading another program's memory

Alternative Concept and Complexity Analysis

If the monitors used polling (checking the sensor in a loop) instead of interrupts, what would the solution be? Evaluate the worst-case delay before an abnormal reading is noticed (the polling interval versus the interrupt latency) and the CPU time and power wasted by polling.

Evidence Expected

A reasoned explanation of CPU modes, interrupts and protection, applied to the monitors.

TASK 9 – Evaluate the Memory Hierarchy and I/O Systems — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how data moves between storage levels and devices.

Discuss:

- The levels of the memory hierarchy (registers, cache, main memory, SSD, hard disk, network storage) with typical access times and sizes
- Where MRI images, patient records and monitor readings should be kept
- Programmed I/O, interrupt-driven I/O and DMA, and which suits the MRI scanner
- The effect of caching on the speed of record look-up

Alternative Concept and Complexity Analysis

If the MRI scanner transferred images with interrupt-driven I/O (one interrupt per word) instead of DMA, what would the solution be? Evaluate the number of interrupts and the CPU time for a 500 MB image, and the memory buffers needed in each method.

Evidence Expected

A justified data-placement plan and I/O method for each device, using typical access times.

TASK 10 – Measure and Compare **4 Marks**

Bloom's Level: K5 – Evaluate

Measure the behaviour of your programs on a Linux system.

Discuss:

- The number of system calls made by your copy program for different buffer sizes (strace -c)
- The time spent in user mode and in kernel mode (the time command)
- The time to create 1,000 processes with fork() and to create 1,000 threads
- How the measurements support or contradict your earlier judgements

Alternative Concept and Complexity Analysis

If threads were created instead of processes for the measurements, what would the result be? Evaluate the creation time for 1,000 threads versus 1,000 processes and the memory used by each, and explain the difference using what each one shares.

Evidence Expected

Measured results in tables, compared with expectations, with explained differences.

TASK 11 – Recommend and Defend the Operating-System Plan **4 Marks**

Bloom's Level: K5 – Evaluate

Recommend the complete operating-system plan for the hospital.

Discuss:

- The type and structure of operating system for each environment
- The services, protection and I/O methods each will use
- The limitations and risks of your plan
- Whether the plan remains suitable for the group of three hospitals

Your recommendation must be based on **evidence from your own analysis and measurements**.

Alternative Concept and Complexity Analysis

If the whole system were moved to virtual machines on two large servers, instead of separate physical machines for each environment, what would the solution be? Evaluate the effect on response time for the monitors, the memory and processor overhead of virtualisation, and the effect on reliability.

Evidence Expected

A technically justified plan supported by analysis and measurements.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design — 8 Marks

Bloom's Level: K5 – Evaluate

Changed Requirement

The hospital adds **remote monitoring devices in patients' homes** that send readings over the internet, and moves patient records to a **cloud service** shared with the partner hospitals.

Re-evaluate your original design. You may **RETAIN** → **MODIFY** → **or REDESIGN** your original solution.

A. **Impact Analysis – 2 Marks**

Identify which operating-system types, services and protection mechanisms are affected.

Evidence Expected

A clear analysis of how the changed requirements affect the original design.

B. **Evaluation of the Original Design – 2 Marks**

Evaluate whether your plan can support home devices and cloud records, considering response time, reliability and security.

Evidence Expected

A technically supported evaluation of the original design.

C. **Adaptation or Redesign – 2 Marks**

Describe the revised plan (for example: an embedded real-time OS on home devices, a distributed system for shared records).

Evidence Expected

An appropriate design response.

D. **Evidence-Based Defence – 2 Marks**

Defend your decision using evidence and the trade-offs involved.

Evidence Expected

A well-supported final decision integrating evidence and the changed requirements.

MARK DISTRIBUTION

Task	Bloom's Level	Marks
Task 1 – Requirements of Each Environment	K5 – Evaluate	4
Task 2 – Types of Operating Systems	K5 – Evaluate	3
Task 3 – Operating-System Structures	K5 – Evaluate	4
Task 4 – Operating-System Services	K5 – Evaluate	4
Task 5 – System Calls	K5 – Evaluate	4
Task 6 – Cost of System Calls	K5 – Evaluate	4
Task 7 – Process Control System Calls	K5 – Evaluate	4
Task 8 – CPU States and Protection	K5 – Evaluate	3

Task	Bloom's Level	Marks
Task 9 – Memory Hierarchy and I/O	K5 – Evaluate	4
Task 10 – Measurement and Comparison	K5 – Evaluate	4
Task 11 – Final Recommendation	K5 – Evaluate	4
Subtotal		**42**
Task 12 – Day-4 Design Challenge	K5 – Evaluate	**8**
TOTAL		**50**

Submission Requirements

- **GitHub repository link (mandatory)** containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.
- Your assignment report containing the required evidence for Tasks 1–12, including requirement tables, diagrams, strace and time outputs, measurements and complexity analyses.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted

Contents

Part	Title	Marks
A	Execution environment, GitHub plan and scenario summary	–
Task 1	Evaluate the requirements of each computing environment	4
Task 2	Judge the type of operating system for each environment	3
Task 3	Evaluate operating-system structures	4
Task 4	Evaluate the services an operating system must provide	4
Task 5	Examine system calls	4
Task 6	Evaluate the cost of system calls (Complete the Code)	4
Task 7	Evaluate process-control system calls (Complete the Code)	4
Task 8	Evaluate CPU states and protection	3
Task 9	Evaluate the memory hierarchy and I/O systems	4
Task 10	Measure and compare	4
Task 11	Recommend and defend the operating-system plan	4
Task 12	Day-4 Design Challenge: re-evaluate and adapt the design	8
End	Submission checklist and viva preparation	–
	Total	**50**

Part A – Execution Environment, GitHub Plan and Scenario

A.1 Execution environment

Every program in this report was compiled and executed on Linux, and every measurement in Tasks 5, 6, 7 and 10 was taken with the tools named below. The numbers are quoted exactly as obtained; they are a single-CPU virtual machine, so absolute times will differ on other hardware but the ratios and counts are what the evaluation depends on.

Item	Value
Operating system	Ubuntu 24.04.5 LTS, Linux kernel 6.18, 1 virtual CPU, 4 KB page size, 8 MB default stack
Compiler	gcc 13.3.0
Compile flags	-std=c11 -Wall -Wextra -pthread (the benchmark uses -O1 -std=gnu11 -pthread)
Tools	strace 6.8 (strace -c for counts), bash time (real/user/sys), clock_gettime(CLOCK_MONOTONIC)

A.2 GitHub repository and commit plan

Repository name: , shared with the course faculty (public, or private with the faculty added as a collaborator). One folder per task, each containing C sources, scripts, outputs and screenshots, plus a README.md. At least one commit is made for each task, so the history shows progressive work.

Folder	Contents	Commit message
Task01 – Task04	Analysis tables (Markdown copies of the report tables), diagrams	Task 1: requirement ranking …
Task05	read_demo.c, strace_read_demo.txt	Task 5: strace of read()
Task06	copy_syscalls.c (completed), output.txt, strace_c.txt	Task 6: complete the code, strace -c
Task07	mini_shell.c, mini_shell_bug.c, output.txt, bug_redirected.txt	Task 7: fork/exec/waitpid shell

Folder	Contents	Commit message
Task08	polling vs interrupt calculation sheet	Task 8: CPU modes and polling
Task09	data-placement table, DMA vs interrupt calculation	Task 9: memory and I/O
Task10	bench.c, copy_times.txt, time_output.txt, proc_thread.txt, strace_*.txt	Task 10: measurements
Task11 – Task12	final plan, Day-4 redesign (Dockerfile / YAML if used)	Task 11/12: final plan and redesign

The README.md contains: name, register number, compiler and version (gcc 13.3.0), distribution and version (Ubuntu 24.04.5 LTS), the exact compile command (gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short result note for each task.

A.3 Scenario summary and assumptions

A 500-bed hospital is replacing its computer systems. The IT team proposes one general-purpose operating system everywhere; the clinical staff fear delayed alarms, database stoppage during updates and short tablet battery life. The environments and growth stages are summarised below and are used throughout the report.

Environment	Code	Scale: 1 ward	Scale: 1 hospital	Scale: group of 3
Bedside patient monitors	E1	40	500	about 1,500
Nurses' handheld tablets	E2	10	120	about 360
Doctors' desktop workstations	E3	a few	80	about 240
Central patient-records database server	E4	1 node	cluster	shared cluster / replicas
MRI imaging server (many processors)	E5	–	1 server	1 per site

Environment	Code	Scale: 1 ward	Scale: 1 hospital	Scale: group of 3
Link to two partner hospitals	E6	–	1 gateway	mesh of 3 sites

Assumptions: the alarm deadline is 10 ms from abnormal reading to alarm; a shift is 8–12 hours; the database must be available 24×7; typical hardware figures quoted from manufacturers' and textbook ranges are labelled "typical", and anything labelled "measured" was obtained on the machine in A.1.

Task 1 – Evaluate the Requirements of Each Computing Environment (4 Marks, K5)

1.1 Requirements of each device

Each environment is judged on five requirements: response time, reliability, throughput, battery/power and security. A rank of 1 is the most important for that environment.

Environment	Rank 1	Rank 2	Rank 3	Rank 4	Rank 5
E1 Bedside monitor	Response time (10 ms hard deadline)	Reliability	Security	Battery (backup)	Throughput
E2 Nurse tablet	Battery (full shift)	Security (device can be lost)	Response (interactive)	Reliability	Throughput
E3 Doctor workstation	Security	Response (interactive)	Reliability	Throughput	Battery (mains)
E4 Database server	Reliability / availability	Security	Throughput (transactions)	Response	Battery (UPS)
E5 Imaging server	Throughput	Reliability	Security	Response (seconds)	Battery (mains)
E6 Partner link	Security	Reliability	Throughput	Response	Battery (mains)

1.2 Justification of the ranking

- **E1 Monitor – response time first.** A missed deadline is a patient-safety failure, not a slow screen. Throughput is lowest because a monitor handles only a few hundred samples per second. Reliability is second because a monitor that hangs gives no alarm at all.

- **E2 Tablet – battery first.** A tablet that dies mid-shift cannot be used for drug administration or charting. Security is second because a lost tablet holds patient data; this makes encryption and remote wipe mandatory.
- **E3 Workstation – security first.** Doctors view and edit full records on a shared desktop, so authentication and access control matter more than speed; battery is irrelevant on mains power.
- **E4 Database – reliability first.** The database "must never stop"; throughput and response matter, but an up-but-corrupt or down database stops the whole hospital.
- **E5 Imaging – throughput first.** MRI reconstruction is a long, CPU-heavy batch job that is judged by images processed per hour; a result delay of seconds is acceptable.
- **E6 Partner link – security first.** Records cross organisational boundaries over a public network, so confidentiality and integrity outrank speed.

1.3 Conflicting requirements

- **Fast response vs many users on one machine:** time-sharing among many users adds scheduling delay, so workstations and the database accept tens of milliseconds, while monitors cannot share the CPU with any non-real-time work.
- **Battery vs response/security:** keeping radios and CPU awake gives quick response and constant sync but drains the tablet; encryption and background scanning cost power.
- **Throughput vs reliability:** aggressive caching and write-back raise throughput but risk data loss in a crash, so the database uses journaling and synchronous commits at some speed cost.
- **Security vs response:** access checks, encryption and audit logs add cycles; on a monitor this would threaten the deadline, so security is moved to the network edge and the device is made small and tamper-resistant.
- **Availability vs updating:** patching normally needs a reboot, conflicting with "never stop"; this is solved by clustering and rolling updates (Task 2).

1.4 Requirements critical to patient safety

Critical to patient safety	Why
Alarm within 10 ms (E1)	A late or lost alarm can delay resuscitation.
Database availability and integrity (E4)	Allergy, drug and history data are needed for every treatment decision.
Protection of one patient's record from another user (E3, E4)	Wrong-patient data or leakage harms patients and breaks the law.
Tablet battery for the full shift (E2)	Bedside medication checks depend on it.
Correct, complete imaging results (E5)	A corrupted MRI can cause a wrong diagnosis; speed is secondary to correctness.

1.5 Lessons from the history of operating systems

Generation	Idea	Where it applies in the hospital
Batch systems	Jobs queued and run without interaction to keep the CPU busy	E5: MRI reconstruction and nightly backups are batch jobs measured by throughput.
Time-sharing / multiprogramming	CPU shared in short slices, giving many users interactive response	E3 and E4: many doctors and clients share the workstation and database server.
Personal-computer systems	One user, convenience and a GUI, protection added later	E3: desktop OS with a GUI; security had to be added (login, permissions, updates).
Mobile / handheld systems	Small memory, power management, touch, apps in sandboxes	E2: tablets need aggressive power management and sandboxed apps.
Real-time systems	Correctness depends on meeting deadlines	E1: monitors need a hard real-time scheduler.
Distributed and clustered systems	Many machines cooperate; failure of one is tolerated	E4 and E6: database fail-over and record sharing between hospitals.

Alternative Concept and Complexity Analysis – one general-purpose OS everywhere

Solution under the alternative. All devices (monitor, tablet, workstation, servers) would run the same general-purpose kernel, for example an unmodified desktop Linux or Windows image. The monitor application would be an ordinary process with the default time-sharing scheduler.

Criterion	Different OS per environment (original plan)	One general-purpose OS everywhere
Worst-case alarm response	Bounded: fixed-priority pre-emptive scheduler, the alarm task runs within a known interrupt latency plus task time, tens of microseconds to under 1 ms (typical)	Unbounded in theory: the alarm process waits behind other runnable processes, page faults, disk I/O and kernel-locked sections; delays of tens to hundreds of milliseconds under load (typical)
Memory on a monitor	RTOS kernel in the tens of KB, application plus buffers in a few hundred KB to a few MB (typical)	Hundreds of MB to GB for kernel, libraries, services and file system (typical)
Administration effort	Several images (RTOS, handheld, desktop, server) to patch and qualify; more skills needed	One image to patch, one skill set; but every patch touches life-critical monitors and needs a reboot

Complexity. Let a monitor have n competing runnable tasks. In the real-time kernel the scheduler picks the highest-priority task in O(1) using a priority bit-map, so the worst-case response is the constant R = L_irq + C_alarm, independent of n. In a general-purpose kernel the pick is O(log n) (red-black tree in Linux CFS), but the wait is not bounded by the scheduler cost: an alarm task of low weight can wait $\Theta(n \times \text{time-slice})$, i.e. R grows with the load and has no guaranteed upper limit. **Space:** the real-time image is O(1) in the number of installed services ($S \approx 1$), the general-purpose image is O(S) with S in the hundreds of services. **Administration:** with k different OS images and P patches per year the effort is O(k

× P) for qualification plus O(d) for rolling out to d devices; the single OS reduces the first term to O(P) but not the second.

Judgement. A single OS cannot give the monitors a guaranteed 10 ms deadline, and its memory footprint does not fit a small embedded board. The reduced administration effort does not outweigh the patient-safety risk, so the single-OS proposal is rejected for E1, and kept only where requirements are similar (workstations and servers can share a Linux distribution).

Evidence

The ranked requirement table (1.1) with its justification (1.2), the conflict list (1.3), the safety-critical list (1.4), the history table (1.5) and the alternative comparison above are the evidence for this task. The latency argument is quantified in Task 8 and the monitor structure in Task 3.

Task 2 – Judge the Type of Operating System for Each Environment (3 Marks, K5)

2.1 Comparison of operating-system types

Type	Key property	Fit for the hospital
Mainframe	Very high reliability, huge I/O throughput, batch plus transactions	Good for the records database in a very large hospital group, but cost is very high; clustered Linux servers give most of the benefit.
Desktop	One interactive user, GUI, time-sharing	Best for doctors' workstations.
Multiprocessor (SMP / AMP)	Several CPUs share memory under one OS	Best for the MRI imaging server.
Distributed	Machines connected by a network, appear as one system or cooperate by messages	Best for the link to partner hospitals and shared records.
Clustered	Nodes share storage or replicate data and take over on failure	Best for the database server (fail-over).
Hard real-time	A missed deadline is a failure; bounded latency	Required for bedside monitor alarms.
Soft real-time	Deadline misses degrade quality but are not fatal	Suitable for display refresh, trend graphs, video of tele-consultation.
Handheld	Small memory, battery, touch, sandboxed apps	Best for nurses' tablets.

2.2 Chosen type for each environment, with rejected alternatives

Environment	Chosen type	Rejected alternatives and reasons
E1 Monitors	Hard real-time OS (RTOS such as QNX or FreeRTOS)	Desktop/general-purpose: no bounded latency. Soft real-time: a missed alarm is unacceptable. Handheld: no deadline guarantee.
E2 Tablets	Handheld OS (Android- or iOS-class, managed by MDM)	Full desktop OS: higher memory, start-up time and power. Real-time OS: no rich apps or app ecosystem and no need for hard deadlines.
E3 Workstations	Desktop OS (Linux or Windows)	Handheld: small screen and weak multitasking for long records work. Mainframe terminal: loses local imaging viewers and flexible software.
E4 Database	Clustered server OS (Linux, 2–3 node active/passive or active/active with fail-over)	Single big multiprocessor: single point of failure (see alternative below). Mainframe: higher cost with little extra benefit at this scale.
E5 Imaging	Symmetric multiprocessing (SMP) Linux with many cores and GPUs	Asymmetric multiprocessing: master CPU is a bottleneck and single point of failure. Real-time OS: deadline is not the issue, throughput is.
E6 Partner link	Distributed system: client–server / service-based exchange over authenticated TLS	Direct database sharing over the internet: no isolation, one site's outage stops the other. Desktop OS alone: no replication or identity federation.

2.3 Symmetric versus asymmetric multiprocessing for the imaging server

In **asymmetric multiprocessing (AMP)** one master processor runs the OS and assigns work to slave processors; it is simple but the master becomes a bottleneck and, if it fails, the whole machine stops. In **symmetric multiprocessing (SMP)** every processor runs the same kernel

and can execute any task, with shared memory and a shared run queue (or per-CPU queues with load balancing). MRI processing divides into many independent slices, so SMP spreads them evenly and keeps working at reduced speed if one CPU is taken offline. The speed-up is limited by the serial fraction as given by Amdahl's law: $S(N) = 1 / ((1 - p) + p / N)$. The cost of SMP is lock contention and cache-coherency traffic as N grows.

2.4 Clustered database server with fail-over

Two or more database nodes are connected by a private heartbeat network and either share storage (SAN) or replicate the data synchronously. If the active node stops sending heartbeats for a timeout (for example 1–3 seconds), the cluster manager promotes the standby, moves the service address and the clients reconnect, so the typical interruption is seconds rather than hours. Updates are applied **node by node (rolling update)**: the standby is patched and rebooted, service is switched to it, then the former active node is patched, so the database never stops for the staff. This directly answers the clinical concern about updates.

Alternative Concept and Complexity Analysis – one large multiprocessor instead of a cluster

Solution under the alternative. The database runs on a single large SMP machine (for example 64 cores and 2 TB of memory) with redundant power supplies but one operating-system instance.

Criterion	Cluster with fail-over	Single large multiprocessor
Availability	No single point of failure; node failure causes a fail-over of seconds. If a node is 99.9 % available, two independent nodes give $1 - (0.001)^2 = 99.9999$ % ideal, about 32 seconds of downtime a year	One OS image is a single point of failure: 99.9 % means about 8.8 hours of downtime a year, and every kernel update needs a full stop
Cost	Several mid-range servers plus cluster software; cost grows roughly linearly with nodes	One large machine; price grows faster than linearly with CPU count and memory (premium for large SMP)

Criterion	Cluster with fail-over	Single large multiprocessor
Throughput vs processors (time)	Scales with nodes if data can be partitioned or replicas serve reads; overhead of replication messages O(n)	Amdahl: with p = 0.95, S(8) = 5.9, S(32) = 12.6, limit 20; lock contention flattens the curve
Memory available (space)	Each node has its own memory; total is the sum but data cached on several nodes is duplicated	One shared address space; a large working set fits in one memory with no duplication, up to the machine's maximum

Complexity. Time: throughput of the SMP is $T(N) = T(1) \times S(N)$ with $S(N) = 1/((1 - p) + p/N)$, which is O(1) bounded by $1/(1 - p)$ for fixed p; the cluster's read throughput is O(N) for N replicas but every write costs O(N) messages to keep replicas consistent. Space: the SMP needs O(D) memory for a data set of size D; the cluster needs $O(D \times r)$ in total for r replicas. **Judgement:** the cluster is chosen because availability, not peak throughput, is the top requirement of E4; the SMP is kept for E5, where throughput dominates and a short stop is tolerable.

Evidence

Tables 2.1 and 2.2 give the justified type per environment with rejected alternatives and reasons, 2.3 and 2.4 discuss SMP/AMP and fail-over, and the availability and Amdahl calculations are shown in the alternative analysis.

Task 3 – Evaluate Operating-System Structures (4 Marks, K5)

3.1 Comparison of structures

Structure	Reliability	Performance	Ease of updating
Simple / monolithic	Low to medium: every driver and service runs in kernel mode, so one bug can crash the whole system	Highest: a service is a direct function call inside the kernel	Hard: usually rebuild or reboot
Layered	Good: each layer uses only the layer below, easy to verify	Lower: every request passes down several layers	Medium: a layer can be replaced if its interface is kept
Microkernel	Highest: only scheduling, memory, IPC in the kernel; drivers and file system are user-mode servers that can be restarted	Lower: services cost message passing and context switches	Easy: servers can be replaced or restarted without a reboot
Modular (loadable kernel modules)	Medium: modules run in kernel mode, a bad module can still crash	High: same as monolithic once loaded	Good: load or unload drivers at run time
Hybrid	Medium to good: mixes a microkernel-style design with some services in kernel for speed	Good	Good

3.2 Which structure suits which environment

- **Patient monitors – microkernel-style real-time OS.** A small trusted kernel with the sensor driver, alarm logic and network stack as separate protected tasks isolates faults. A faulty network driver can be restarted without stopping the alarm task, and the small kernel can be analysed for worst-case timing.

- **Database server – modular monolithic kernel (Linux).** The database is judged on throughput; running the file system, block drivers and TCP/IP inside the kernel gives system calls at function-call speed, and loadable modules allow driver and file-system updates without a rebuild. Fail-over (Task 2) covers the reliability weakness.

3.3 How real operating systems are structured

Operating system	Structure	Notes
Linux	Monolithic kernel with loadable kernel modules	Drivers, file systems, networking in kernel space; modules loaded on demand.
Windows (NT family)	Hybrid	Microkernel-inspired executive and HAL, but most services and drivers run in kernel mode for speed.
Android	Layered on a Linux kernel	Linux kernel, hardware abstraction layer, native libraries and runtime, application framework, apps; each app is sandboxed in its own process.
FreeRTOS	Very small kernel linked with the application	Scheduler, queues, semaphores, timers; tasks share one address space unless the optional MPU port is used.
QNX	True microkernel RTOS	Kernel offers scheduling, IPC and interrupts; drivers, file systems and networking are user-space processes communicating by messages; a failed driver is restarted.

Alternative Concept and Complexity Analysis – monolithic kernel on the monitors

Solution under the alternative. The bedside monitor would run a monolithic kernel in which the sensor, display and network drivers are linked into one kernel image and executed in kernel mode.

Criterion	Microkernel (original)	Monolithic (alternative)
Time cost of a service request	App → kernel → server → kernel → app: at least two messages and two context switches; typically a few microseconds	One system call (trap, kernel function, return); typically a fraction of a microsecond to about a microsecond
Kernel memory	Kernel in the tens of KB; each server is a separate small process	Kernel image of hundreds of KB to many MB with all drivers resident
Effect of a faulty driver	Driver process crashes and is restarted; the alarm task keeps running	Kernel panic or memory corruption: the entire monitor stops, including the alarm

Complexity. Let c_trap be the cost of one trap and c_ipc the cost of one message, with a request passing through s servers. Monolithic request time is O(1) = c_trap. Microkernel request time is O(s) = 2 × s × c_ipc, because each server hop needs a request and a reply. Both are constants compared with the 10 ms deadline: our measurements in Task 10 give about 0.2 µs per system call on the test machine (134 million calls in 30.7 s), so even a 10× slower microkernel path of a few microseconds leaves more than 99.9 % of the deadline unused. Kernel space is O(D) for the monolithic kernel (all D drivers resident) and O(1) + O(loaded servers) for the microkernel. Fault containment is O(1) blast radius (one server) versus the whole system.

Judgement. The small speed penalty of the microkernel is acceptable because the deadline is dominated by interrupt latency and task execution, not by system-call cost, while the fault isolation is a real patient-safety gain. The monolithic structure is retained for the database server and imaging server.

Evidence

The structure comparison table (3.1), the defended choices for two environments (3.2), the real-OS structures (3.3) and the cost comparison above. The system-call cost used in the complexity paragraph is measured in Tasks 6 and 10.

Task 4 – Evaluate the Services an Operating System Must Provide (4 Marks, K5)

4.1 Service table for each environment

Legend: **Full** = complete service, **Min** = minimal or reduced, **No** = removed.

Service	E1 Monitor	E2 Tablet	E3 Work-station	E4 Database	E5 Imaging	E6 Link
User interface	Min (small display, buttons)	Full touch GUI	Full GUI	No GUI (remote admin CLI)	No GUI (CLI/batch)	No
Program execution	Full (fixed tasks)	Full (sandboxed apps)	Full	Full	Full	Full
I/O operations	Full (sensor, display)	Full	Full	Full (storage heavy)	Full (DMA)	Network
File system	Min (small flash log)	Full	Full	Full (journaled)	Full (large files)	Min
Communication	Min (secure link to nurses' station)	Wi-Fi, secure	Full	Full	Full	Full (TLS/VPN)
Error detection	Full (watchdog, sensor fault)	Full	Full	Full (ECC, RAID, checksums)	Full	Full
Resource allocation	Fixed priorities	Power-aware	Full	Full	Full (many CPUs)	Full

Service	E1 Monitor	E2 Tablet	E3 Work-station	E4 Database	E5 Imaging	E6 Link
Accounting	No	Min	Min	Full (audit)	Min	Full (audit)
Protection and security	Min (MPU, signed firmware)	Full (encryption, MDM)	Full	Full	Full	Full

4.2 Services removed from the bedside monitor

To make the monitor smaller and faster, the following are removed: the general user interface and shell, multi-user accounts, a general file system (only a small circular log in flash is kept), accounting, virtual memory and swapping (swapping would add unbounded delay), dynamic loading of programs, printing, and any network service that is not needed. Smaller code means fewer bugs, less memory, faster start-up and a smaller attack surface.

4.3 Essential services for the database server

- **File system** with journaling and checksums so that a crash never leaves records half-written.
- **Protection and security:** authentication, access control, encryption at rest and in transit, and audit logging.
- **Resource allocation:** memory and I/O scheduling so a heavy report cannot starve patient look-ups.
- **Error detection and recovery:** ECC memory, RAID, fail-over and backup.
- **Communication:** a reliable network stack for clients and replication.
- **Accounting:** who accessed which record, for legal audit.

4.4 How the OS protects one patient's records from another user's programs

1. **Authentication** identifies each user (password plus second factor, smart card); every process runs on behalf of a user ID.
2. **Memory protection:** hardware (MMU, base/limit or page tables) gives each process its own address space, so one program cannot read another's memory (Task 8).
3. **File access control:** owner, group and permission bits or access-control lists; role-based rules (doctor, nurse, billing) and mandatory access control such as SELinux limit a process even when it runs as the right user.
4. **Mode separation:** only the kernel can touch the disk; programs must ask through system calls, where permission is checked on every open() and read().
5. **Database-level control:** row-level security so a user sees only patients assigned to them, with "break-glass" emergency access that is logged.
6. **Encryption and audit:** records are encrypted on disk, and every access is written to an append-only audit log.

Alternative Concept and Complexity Analysis – full desktop OS on the tablets

Solution under the alternative. Nurses' tablets would run a full desktop operating system with every service enabled (desktop shell, indexing, printing, multi-user accounts, background updates).

Criterion	Reduced handheld OS (original)	Full desktop OS (alternative)
Memory and storage (space)	About 2–4 GB RAM and a few GB of system storage in use (typical)	About 4–8 GB RAM and tens of GB of storage (typical)
Start-up time (time)	Seconds to tens of seconds	Tens of seconds to minutes, because many more services are started

Criterion	Reduced handheld OS (original)	Full desktop OS (alternative)
Battery life (time)	A full 8–12 hour shift (typical)	Noticeably shorter: more background processes keep the CPU and radios awake

Complexity. If the OS runs S services, memory is $M = M_{kernel} + \Sigma m_i$, which is O(S); start-up time is $T_{boot} = \Sigma t_i$ for serial start-up, O(S). Battery life $L = E / (P_{base} + \Sigma p_i)$ decreases as services are added: with E = 40 Wh and P = 3 W the life is 13 hours, but adding 2 W of background services gives 8 hours, which is less than a shift. **Judgement:** the reduced handheld OS is kept; the desktop OS only wins on application compatibility, which nurses' tablets do not need.

Evidence

The service table (4.1) lists inclusions and exclusions for every environment; 4.2–4.4 justify them; the alternative comparison and the battery calculation show the cost of the full desktop OS.

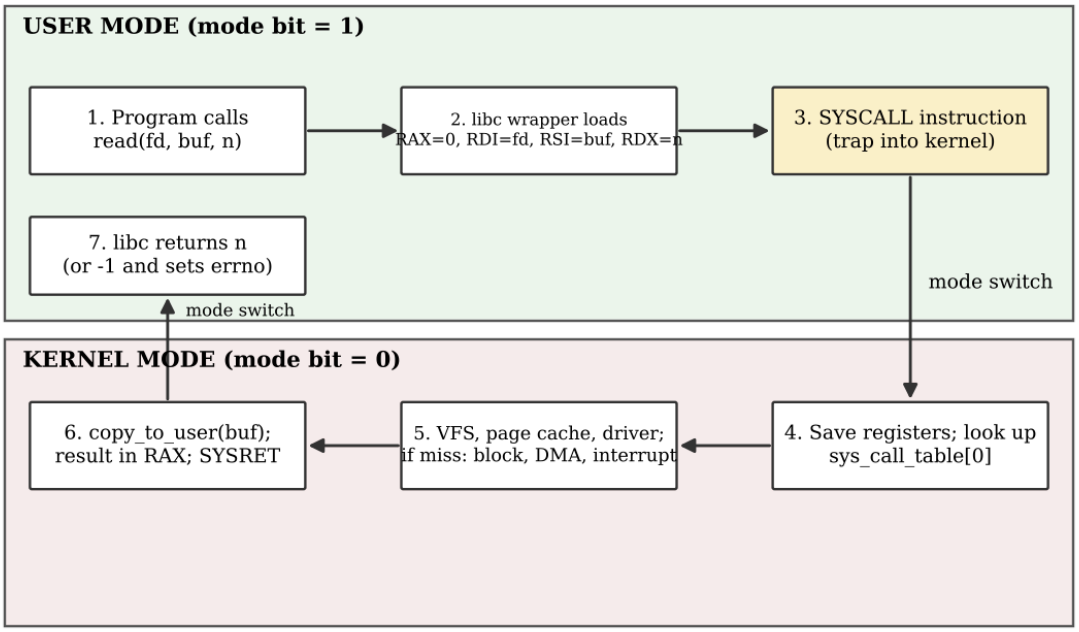
Task 5 – Examine System Calls (4 Marks, K5)

5.1 Categories of system calls

Category	Purpose	Linux examples
Process control	Create, run, wait for, end and signal processes; allocate memory	fork, execve, wait4/waitpid, exit_group, kill, brk, mmap
File management	Create, open, read, write, close, delete and seek files	open/openat, read, write, close, lseek, unlink, stat
Device management	Request, use and release devices	ioctl, read/write on /dev files, mmap of device memory
Information maintenance	Get or set system and process information	getpid, time/clock_gettime, uname, sysinfo, getrusage
Communication	Exchange data between processes or machines	pipe, socket, connect, send, recv, shmget, msgsnd
Protection	Control access to resources	chmod, chown, umask, setuid, setgid

5.2 What happens step by step when a program calls read()

Figure: Step-by-step execution of the read() system call



1. **Library call.** The program calls read(fd, buf, n), an ordinary C-library function. The wrapper in libc places the system-call number (0 for read on x86-64) in register RAX and the arguments in RDI (fd), RSI (buf) and RDX (n).
2. **Trap.** The wrapper executes the SYSCALL instruction. This is a software trap: the CPU saves the user instruction pointer and flags and jumps to the kernel entry point defined at boot.
3. **Mode switch.** The hardware changes the mode bit from user mode to kernel mode, so privileged instructions and all memory are now accessible.
4. **Kernel work.** The kernel saves the remaining registers, uses RAX to index the system-call table, checks that fd is valid and that the process has permission, and looks in the page cache. If the data is cached it is copied at once; if not, the kernel asks the device driver to start a transfer, blocks the process, runs another process, and is woken by the completion interrupt (DMA moves the data into memory).
5. **Return.** The kernel copies the data to the user buffer (copy_to_user), puts the result (bytes read, or a negative error code) in RAX, restores registers and executes SYSRET, which switches the mode bit back to user mode.
6. **Library return.** libc converts a negative result into -1 and sets errno, and returns n to the program.

5.3 How parameters are passed to the kernel

Method	How it works	Used by
Registers	Number and arguments are placed directly in CPU registers; fastest, but limited by the number of registers (6 arguments on x86-64 Linux)	Linux x86-64 and ARM64
Block (table) in memory	The arguments are stored in a block of memory and its address is passed in a register; allows any number or size of arguments	Used when there are many arguments, and in some Solaris/older Linux interfaces
Stack	The program pushes the arguments on the stack and the kernel pops them	Some BSD and older 32-bit conventions

5.4 strace evidence from a real program

The program read_demo.c opens /etc/hostname, reads it, writes it to the screen and closes it. It was compiled with gcc -std=c11 -Wall -Wextra and run under strace; the lines that correspond to the program's own calls are:

```
openat(AT_FDCWD, "/etc/hostname", O_RDONLY) = 3
read(3, "vm\n", 64)                     = 3
write(1, "vm\n", 3)                     = 3
close(3)                                = 0
exit_group(0)                           = ?
+++ exited with 0 +++
```

The line read(3, "vm\n", 64) = 3 shows the system call (read), its arguments (descriptor 3, buffer, 64 bytes requested) and its return value (3 bytes obtained), exactly the register contents described in 5.2. strace works by using the ptrace facility to stop the program at every system-call entry and exit, which is why programs run much slower under strace.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
```

```
#include <unistd.h>
int main(void) {
   char buf[64];
   int fd = open("/etc/hostname", O_RDONLY);      /* open() system call   */
   ssize_t n = read(fd, buf, sizeof buf);         /* read() -> trap       */
   write(STDOUT_FILENO, buf, (size_t)n);          /* write() system call  */
   close(fd);                                     /* close() system call  */
   return 0;
}
```

Alternative Concept and Complexity Analysis – fopen/fread/fwrite instead of open/read/write

Solution under the alternative. The program uses the C library's buffered stream functions. fread() fills an internal buffer of 4,096 bytes (the file system block size reported by the kernel, in our measurement) with a single read() and then serves the program's requests from memory; fwrite() accumulates data and issues one write() when the buffer is full.

```
/* alternative: buffered copy */
FILE *in = fopen(src, "r"), *out = fopen(dst, "w");
char b[4096]; size_t n;
while ((n = fread(b, 1, sizeof b, in)) > 0)
   fwrite(b, 1, n, out);
fclose(in); fclose(out);
```

Number of system calls for a file of n bytes: library buffering gives about n / 4096 read() calls whatever size the program asks for; direct system calls with a buffer of b bytes give n / b. Measured for a 64 MiB file (n = 67,108,864): strace -c counted 16,386 read() calls for the stdio copy: 16,384 data reads (= n / 4096), one end-of-file read and one read made by the dynamic loader. The direct copy with b = 4096 produced the identical 16,386, showing that library buffering behaves like a 4 KB direct buffer. With b = 1 the direct version needs 67,108,864 read() calls, 4,096 times as many. **Extra buffer space:** each open stream allocates a library buffer (4 KB here) plus the FILE structure, so 1,000 open files cost about 4 MB; direct calls use only the program's own buffer.

Complexity. Time (system calls): stdio O(n / B) with B = 4096 fixed; direct O(n / b) chosen by the programmer; both are O(n) in the data size with different constants, and O(1) per-byte cost is only reached when b is large. Space: stdio O(B) per open file; direct O(b) per program. The library is therefore the better default for programs that make small reads (such as reading a text file line by line), and direct calls are better when the program already works in large blocks, as the backup program does.

Evidence

The step-by-step description and diagram (5.2), the parameter-passing comparison (5.3), the strace output of a real program (5.4) and the measured counts of Task 6 (strace_stdio.txt and strace_rw4096.txt in the repository).

Task 6 – Evaluate the Cost of System Calls (4 Marks, K5)

6.1 Why a system call costs more than a function call

An ordinary function call is a jump and return inside one address space, costing a few nanoseconds. A system call must: save the user state; execute a trap that is slower than a call instruction; switch to kernel mode and to the kernel stack; validate every argument (including checking user pointers); possibly run security mitigations that flush or change address-translation state; do the kernel work; and restore state and return. The kernel entry also disturbs the CPU caches, branch predictors and TLB, so the program runs slower for a while after the call. Our measurement gives an average of about 0.23 µs per read() or write() even when the kernel only copies one byte (30.67 s for 2 × 67,108,864 calls), roughly 50 to 100 times the cost of a typical function call. The cost is small once but decisive when multiplied by tens of millions of calls.

6.2 Effect of buffer size on the number of read() and write() calls

For a file of n bytes copied with a buffer of b bytes, the program makes ceil(n / b) calls that return data plus one final call that returns 0, so n / b + 1 read() calls, and the same number of write() calls minus one. For the 10,000-byte test file in the Complete-the-Code program:

Buffer size b	Formula ceil(10000/b)+1	Program output (measured)
1 byte	10,001	10,001
512 bytes	20 + 1 = 21	21
4,096 bytes	3 + 1 = 4	4

6.3 Complete the Code – TODO 1

TODO 1 was replaced by a loop that reads up to bufsize bytes, counts every read() call (including the final one that returns 0), stops at end-of-file or error, and writes exactly the n bytes that were read (with an inner loop in case write() is partial). A guard limits bufsize to the size of buf[] so the array can never overflow. The rest of the program is unchanged.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
```

```
#include <unistd.h>

/* Copy src to dst with the read() and write() system calls, using a buffer of
   bufsize bytes. Returns the number of read() calls made, or -1 on error. */
long copy_file(const char *src, const char *dst, size_t bufsize) {
    char buf[8192];
    int in = open(src, O_RDONLY);
    if (in < 0) return -1;
    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) { close(in); return -1; }
    long calls = 0;
    ssize_t n;
    /* TODO 1 completed: loop of read() / write() */
    if (bufsize > sizeof buf) bufsize = sizeof buf;   /* never overrun buf[] */
    while (1) {
        n = read(in, buf, bufsize);   /* one read() system call */
        calls++;                      /* count every read call, including the last one (returns 0) */
        if (n <= 0) break;            /* 0 = end of file, -1 = error */
        ssize_t done = 0;
        while (done < n) {            /* write exactly the n bytes that were read */
            ssize_t w = write(out, buf + done, (size_t)(n - done));
            if (w < 0) { close(in); close(out); return -1; }
            done += w;
        }
    }
    close(in);
    close(out);
    return calls;
}

int main(void) {
    FILE *f = fopen("records.dat", "w");
    /* a 10,000-byte test file */
    for (int i = 0; i < 10000; i++) fputc('A' + i % 26, f);
```

```
    fclose(f);

    size_t sizes[] = {1, 512, 4096};
    for (int i = 0; i < 3; i++)
        printf("buffer %4zu bytes: %5ld read() calls\n", sizes[i],
               copy_file("records.dat", "copy.dat", sizes[i]));
    unlink("records.dat");
    unlink("copy.dat");
    return 0;
}
```

Compiled with gcc -std=c11 -Wall -Wextra -pthread copy_syscalls.c (no warnings) and executed on Linux. **Output obtained (identical to the expected output):**

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

6.4 Comparison with strace -c

The program was then run with strace -c -e trace=read,write,openat ./copy_syscalls:

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 51.76    0.008024           0     10027           write
 47.65    0.007386           0     10027           read
  0.59    0.000092          10         9           openat
------ ----------- ----------- --------- --------- ----------------
100.00    0.015502           0     20063           total
```

strace reports 10,027 read() calls and 10,027 write() calls. The program printed 10,001 + 21 + 4 = 10,026 read() calls; the one extra read() is made by the dynamic loader at start-up (it reads

the ELF header of libc). The write() calls are accounted for as follows: 10,000 + 20 + 3 = 10,023 writes by the three copies (one fewer than the reads in each copy, because the last read returns 0 and nothing is written), plus 3 writes made by stdio when the 10,000-byte test file is created (4,096 + 4,096 + 1,808 bytes), plus 1 write that flushes the three printf lines at exit (stdout is a pipe here, so it is fully buffered): 10,023 + 3 + 1 = 10,027. The strace count therefore confirms the program's own count exactly, and the extra calls come only from start-up and from the library's buffered calls.

6.5 How library buffering reduces the number of system calls

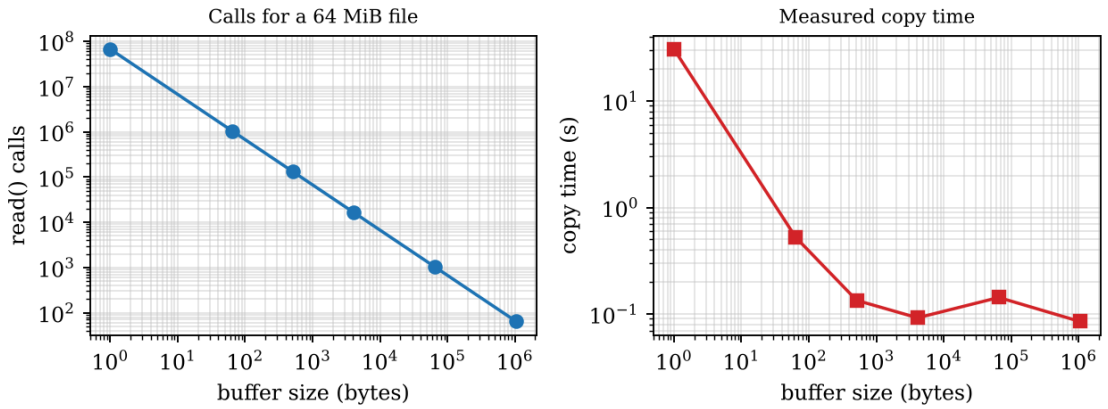
When a program calls fputc or fwrite for a few bytes, the C library stores them in a user-space buffer and calls write() only when the buffer (4 KB for files) is full or on fflush/fclose. The test file creation in this very program makes 10,000 fputc calls but only 3 write() system calls, a 3,333-fold reduction. The same applies to fread.

6.6 Best buffer size for the backup program (from measurements)

A 64 MiB file was copied with the direct read()/write() method for different buffer sizes (benchmark program bench.c in Task10; file system cache warm, one virtual CPU):

Buffer size	read() calls	Copy time, run 1 (s)	Remarks
1 byte	67,108,864	30.673	333 times slower than 4 KB
64 bytes	1,048,576	0.524	5.7 times slower than 4 KB
512 bytes	131,072	0.134	about 1.5 times slower
4,096 bytes (page size)	16,384	0.092	knee of the curve
65,536 bytes (64 KiB)	1,024	0.143 (run 1), 0.093 (run 2)	run-to-run noise larger than the difference with 4 KiB
1,048,576 bytes (1 MiB)	64	0.085	gain below 8 % over 4 KiB

Figure: Effect of buffer size on number of system calls and time



Recommendation. Time falls steeply until about 4 KB (about 99.7 % less than with 1 byte) and then flattens: from 4 KB to 1 MiB the number of calls drops 256-fold, yet the time improves by less than 10 %, because the cost is by then dominated by copying and disk-cache work, not call overhead. For the hospital's backup program I recommend **64 KiB**: it uses 16 times fewer system calls than 4 KiB (1,024 instead of 16,384 for 64 MiB), costs only 64 KiB of memory per copy stream, fits in the CPU's L2 cache, and matches the larger transfer sizes that disks and network storage prefer. Larger buffers (1 MiB and more) add memory per stream and cache pollution for no measurable benefit.

Alternative Concept and Complexity Analysis – memory-mapped files (mmap)

Solution under the alternative. The backup maps the source and destination files into its address space with mmap() and copies with memcpy(); the kernel loads pages on demand through page faults.

```
int in = open(src, O_RDONLY); fstat(in, &st);
int out = open(dst, O_RDWR|O_CREAT|O_TRUNC, 0644); ftruncate(out, st.st_size);
char *a = mmap(0, st.st_size, PROT_READ,  MAP_PRIVATE, in,  0);
char *b = mmap(0, st.st_size, PROT_WRITE, MAP_SHARED,  out, 0);
memcpy(b, a, st.st_size);
munmap(a, st.st_size); munmap(b, st.st_size);
```

Criterion	read()/write() with buffer b	mmap
Number of system calls	about n / b (measured 16,384 reads for 64 MiB at b = 4 KB)	a few: open, fstat, ftruncate, mmap, munmap, close (measured with strace -c: no read() or write() for the copy itself; 10 mmap, 3 munmap and 1 ftruncate calls, including those of program start-up)
Copying between kernel and user space	Two copies: kernel to user buffer, then user buffer to kernel	One copy (memcpy between two mapped regions); no kernel-to-user copy of the file
Memory for very large files	Only b bytes of user buffer, whatever the file size	Virtual address space of n bytes per mapping (limited on 32-bit systems); physical memory used only for resident pages, and I/O errors or truncation appear as SIGBUS
Measured time (64 MiB)	0.092 s (b = 4 KB)	0.071 s (run 1), 0.089 s (run 2)

Complexity. System calls: O(n / b) for read/write versus O(1) for mmap. The page-fault count of mmap is O(n / P) with page size P = 4 KB, so the fault overhead replaces the system-call overhead and the saving is much smaller than the system-call counts suggest (measured about 20 % faster). Data copies: 2n bytes moved versus n bytes. Space: O(b) versus O(n) virtual and O(resident set) physical. **Judgement:** mmap is attractive for random access to large files, but for a sequential backup of huge files a 64 KiB read/write loop is simpler, uses bounded memory and handles read errors cleanly, so it is kept.

Evidence

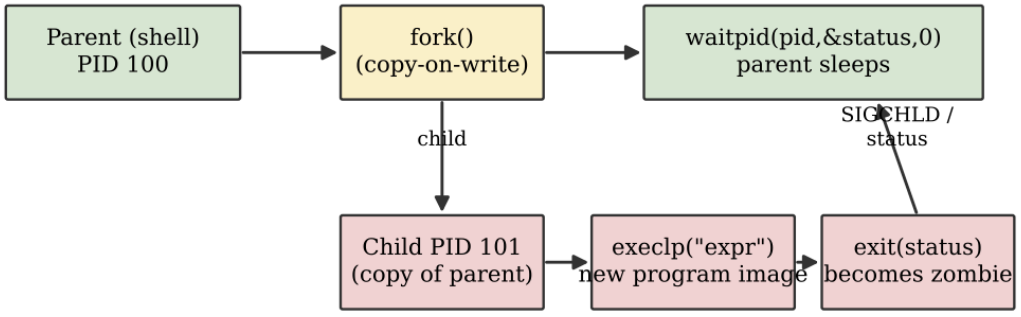
Measured system-call counts and times for six buffer sizes and for stdio and mmap (6.6), the strace -c comparison (6.4), the completed code and its output (6.3), and a justified recommendation (64 KiB).

Task 7 – Evaluate Process Control System Calls (4 Marks, K5)

7.1 What fork(), exec(), wait() and exit() do

Call	Action
fork()	Creates a new process that is a copy of the caller (same code, data and open files; memory is shared copy-on-write). Returns the child's PID to the parent and 0 to the child.
exec() family (execlp, execve …)	Replaces the calling process's program image with a new program; the PID and open files stay. It does not return unless it fails.
wait() / waitpid()	Blocks the parent until a child ends and collects its termination status; this also removes the child's process-table entry.
exit() / _exit()	Ends the process with a status code. exit() first runs atexit handlers and flushes stdio buffers; _exit() ends immediately without flushing.

Figure: Process control - fork(), exec(), wait(), exit()



Parent resumes: WEXITSTATUS(status) -> "job 1 exit status 0"; kernel removes the zombie

7.2 How a shell runs a command and obtains its exit status

The shell reads a command line, calls fork(), and in the child (which sees fork() return 0) optionally rearranges file descriptors for redirection and pipes, then calls exec to turn itself into the requested program. The parent shell calls waitpid() and sleeps. When the program ends, it calls exit(status); the kernel keeps the status in the child's process-table entry and

signals the parent. waitpid() returns it in an integer, and WEXITSTATUS(status) extracts the low 8 bits, which the shell stores as $? (0 means success).

7.3 Zombie and orphan processes

- **Zombie:** a child that has ended but whose parent has not yet called wait(). Its memory is released, but the process-table entry with the exit status stays so the parent can collect it. Many zombies can exhaust the process table. The OS removes a zombie when the parent calls wait(), or when the parent itself ends.
- **Orphan:** a child that is still running after its parent has ended. The kernel re-parents it to the init/systemd process (PID 1), which calls wait() on its adopted children, so an orphan is cleaned up properly when it finishes.

7.4 Why fork() followed by exec() rather than a single create-process call

Separating the two steps lets the child change its own environment between them: redirect standard input and output, set up pipes, change directory, set resource limits, drop privileges and reset signals, using the ordinary system calls dup2, chdir, setrlimit and so on, with no need for a huge create-process call with dozens of parameters. Because fork() uses copy-on-write, the copy is cheap, and the child's exec() then discards the copy. The model is simple, flexible and has lasted for decades; its only cost is the page-table copy, measured in Task 10.

7.5 Complete the Code – TODO 1

TODO 1 was completed with fork(), execlp("expr", "expr", "6", "*", "7", NULL) with exit(127) if exec fails, and waitpid() followed by printing job 1's status with WEXITSTATUS. The second job's code, which was damaged in the pasted assignment text, was restored in its intended order (fork; the child calls _exit(3); the parent calls waitpid and prints the status).

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
  pid_t pid;
```

```
    int status;

    printf("shell: running job 1\n");
    fflush(stdout);                 /* empty the buffer before fork() */
    /* TODO 1 completed */
    pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {                 /* child: become "expr 6 * 7" */
        execlp("expr", "expr", "6", "*", "7", (char *)NULL);
        exit(127);                  /* only reached if exec fails */
    }
    waitpid(pid, &status, 0);       /* parent: wait for the child */
    printf("job 1 exit status %d\n", WEXITSTATUS(status));

    printf("shell: running job 2\n");
    fflush(stdout);
    pid = fork();
    if (pid == 0) _exit(3);         /* a child that fails with code 3 */
    waitpid(pid, &status, 0);
    printf("job 2 exit status %d\n", WEXITSTATUS(status));
    return 0;
}
```

Compiled with gcc -std=c11 -Wall -Wextra mini_shell.c (no warnings). **Output obtained (identical to the expected output):**

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

7.6 Why stdout is flushed before each fork()

stdout is line-buffered on a terminal but **fully buffered** when it is redirected to a file or pipe. Anything still in the buffer when fork() is called is copied into the child's memory along with the rest of the data, so both the parent and the child would later flush the same text and it would appear twice. fflush(stdout) empties the buffer before the fork so that the child starts with nothing pending.

7.7 Experiment: remove the second fflush(stdout), change _exit(3) to exit(3), redirect to a file

The variant mini_shell_bug.c was compiled and run with ./mini_shell_bug > bug_redirected.txt. The file contains:

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

Explanation. With the output redirected, stdout is fully buffered, so after "job 1 exit status 0" and "shell: running job 2" are printed they are only in the parent's buffer (not in the file). Because the second fflush was removed, fork() copies this buffer into the child. The child then calls **exit(3)**, which, unlike _exit(3), flushes the stdio buffers; so the child writes the copied text "job 1 exit status 0" and "shell: running job 2" to the file. Later the parent flushes its own copy at normal program end and writes the same lines again. As a result "shell: running job 2" (and also "job 1 exit status 0") appears twice. With _exit(3) the child would end without flushing, and with the fflush the buffer would have been empty, so either of the two original lines prevents the duplication.

Alternative Concept and Complexity Analysis – posix_spawn() or threads instead of fork() + exec()

Solution under the alternative. posix_spawn() creates a new process running a given program in a single call (internally using vfork-style creation, so the parent's address space is

not copied); the file-descriptor changes are described in an attributes structure instead of code run in the child. A thread-based shell would run each command as a function in a new thread of the same process.

```
pid_t pid;
char *argv[] = { "expr", "6", "*", "7", NULL };
posix_spawnp(&pid, "expr", NULL, NULL, argv, environ);
waitpid(pid, &status, 0);
```

Criterion	**fork() + exec()**	**posix_spawn() / vfork**	**Threads**
Time to start 1,000 (measured, creation + wait)	49–82 ms (about 61 µs each)	9.1–10.1 ms, vfork (about 10 µs each)	7.8–8.8 ms (about 8 µs each)
Memory copied or shared	Page tables copied, pages shared copy-on-write, then discarded by exec	Nothing copied; parent suspended until exec	Everything shared (code, data, heap); only a new stack and register set
Isolation between commands	Full: separate address space, own file table, crash affects only the child	Full once exec has run	None: a bug or crash in one command can corrupt or kill all

Complexity. fork() costs O(m) where m is the number of memory pages or page-table entries of the parent (large database processes make fork slower; with copy-on-write, later writes add O(w) page copies for w written pages). posix_spawn and vfork cost O(1) in the parent's size. Thread creation is O(1) with an additional stack of constant size. Starting k commands is therefore O(k × m), O(k) and O(k) respectively. The shell must keep isolation between commands, so fork + exec (or posix_spawn) is kept and threads are rejected for running commands, even though they are about 7 times cheaper to create.

Evidence

A working program that runs commands like a shell with its output (7.5), the process diagram (7.1), the explanation of buffering with the actual redirected output (7.7) and the measured start-up times for the three methods (Task 10).

Task 8 – Evaluate CPU States and Protection (3 Marks, K5)

8.1 User mode, kernel mode, privileged instructions and the mode bit

The CPU has a **mode bit** (in the status register): 1 = user mode, 0 = kernel mode. In user mode the CPU refuses **privileged instructions**, such as those that perform I/O directly, change the page-table base register, set the timer, disable or enable interrupts (CLI/STI) and halt the CPU; trying one raises an exception and the OS terminates the program. Only the kernel runs in kernel mode. The mode changes to kernel mode on an interrupt, exception or system call, and back to user mode by a return-from-trap instruction. This ensures that applications can only reach hardware by asking the kernel.

8.2 Interrupts, traps and the timer interrupt

Event	Type	Cause	Example in the hospital
Interrupt	Asynchronous, from hardware	A device needs attention	Sensor data ready; network packet arrives; MRI DMA completes
Trap / exception	Synchronous, from the running instruction	System call, page fault, divide by zero, illegal instruction	A program calls read(); a bug dereferences a bad pointer
Timer interrupt	Hardware interrupt at fixed intervals	Timer counts down and fires	Gives the CPU back to the kernel so it can pre-empt a looping program or run a periodic task

The timer is essential for protection: because only the kernel may set the timer, no program can keep the CPU forever; when the time slice ends the interrupt returns control to the scheduler. On the monitors, a periodic timer interrupt also drives the sampling and the watchdog.

8.3 What would happen if any program could disable interrupts

A program that executes CLI and then loops, or simply forgets to re-enable interrupts, stops the timer and all device interrupts. On a monitor the sensor interrupts would never be handled,

so an abnormal reading would not be noticed and no alarm would be raised; the display, network and watchdog would stop; the scheduler could not pre-empt, so the whole monitor would effectively freeze. In general, one faulty or malicious program would be able to take over the machine. This is why disabling interrupts is a privileged instruction and why even real-time kernels disable interrupts only for a few microseconds in short, analysed critical sections.

8.4 How the hardware stops one program reading another program's memory

The memory-management unit (MMU) translates every address the program uses through a page table that is owned by the kernel and loaded in a privileged register. Each process has its own page table, so its virtual addresses can only map to its own physical pages, and every page entry carries permission bits (read, write, execute, user/supervisor). An access outside the mapped pages, or a write to a read-only page, raises a trap (a segmentation fault), and the kernel stops the offending process. Simpler hardware uses **base and limit registers**: an address is valid only if base ≤ address < base + limit. Because loading the page-table register or the base/limit registers is privileged, a program cannot widen its own access. Small monitor processors without an MMU often use a memory-protection unit (MPU) to protect the alarm task's memory from the display and network tasks.

Alternative Concept and Complexity Analysis – polling instead of interrupts on the monitors

Solution under the alternative. The monitor's main loop reads the sensor every T milliseconds: while (1) { r = read_sensor(); if (abnormal(r)) alarm(); sleep_until_next_poll(); }. No interrupt handler is used.

Criterion	Interrupt-driven	Polling every T
Worst-case delay before an abnormal reading is noticed	Interrupt latency L_irq: typically 5–50 µs on an RTOS (typical)	Up to T, because the abnormal reading may appear just after a poll; plus loop execution time

Criterion	Interrupt-driven	Polling every T
CPU time	Work only when an event occurs; CPU can sleep between events	Cost c per poll every T: utilisation c / T, e.g. c = 2 µs and T = 1 ms gives 0.2 %, T = 100 µs gives 2 %
Power	Processor sleeps (microamps) between events	Processor must wake every T, so it cannot reach deep sleep; battery backup lasts much less

Complexity. Interrupt-driven: worst-case detection delay D = L_irq = O(1) and independent of the polling period, CPU work O(e) for e events. Polling: D = T + c = O(T), CPU work O(t / T) over a time t, whether or not anything happens. To meet the 10 ms deadline with an alarm path of about 2 ms, polling needs T ≤ 8 ms, and with a safety factor of two T ≈ 4 ms or less; any shorter period raises CPU use and power. With 1 ms polling and the same 10 ms budget the margin is adequate but every one of the 500 monitors would never sleep. **Judgement:** polling meets the deadline only by wasting power and CPU and by leaving a delay that is hundreds of times larger than the interrupt latency; interrupts are retained, with a watchdog timer as a safety net.

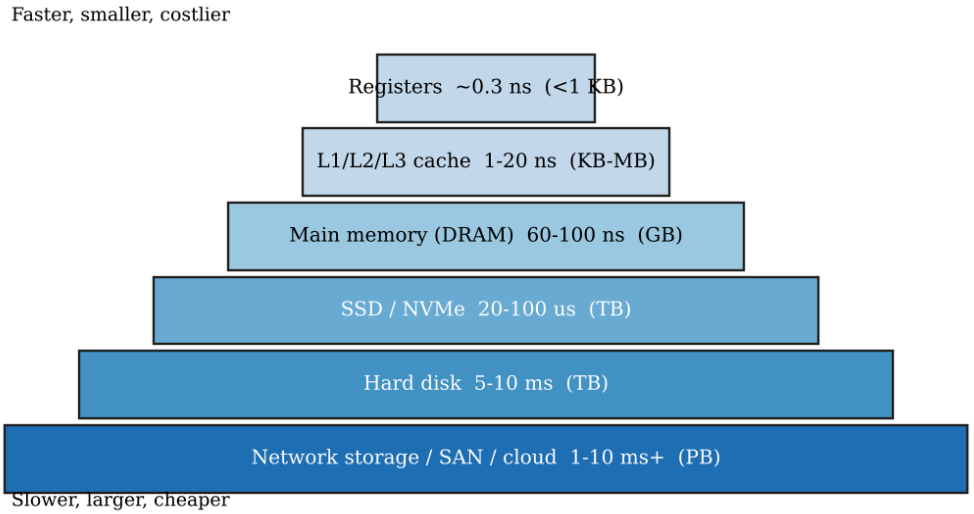
Evidence

A reasoned explanation of CPU modes, interrupts and protection (8.1–8.4) applied to the monitors, with the delay and CPU-utilisation calculation for polling versus interrupts.

Task 9 – Evaluate the Memory Hierarchy and I/O Systems (4 Marks, K5)

9.1 Levels of the memory hierarchy

Figure: Memory hierarchy



Level	Typical access time	Typical size	Volatile?
Registers	about 0.3 ns	under 1 KB	Yes
Cache (L1/L2/L3)	1–20 ns	32 KB – 64 MB	Yes
Main memory (DRAM)	60–100 ns	8 GB – 512 GB	Yes
SSD / NVMe	20–100 µs	0.5 – 8 TB	No
Hard disk	5–10 ms	1 – 20 TB	No
Network storage (SAN/NAS/cloud)	0.5 – 10 ms or more	tens of TB to PB	No

Each level is about 10 to 1,000 times slower, larger and cheaper per byte than the one above; programs work well because of locality of reference, which lets a small fast level hold most of what is needed.

9.2 Where each kind of data should be kept

Data	Placement	Reason
Monitor readings (live)	On-device RAM ring buffer, copied in batches to local flash and streamed to the central server	Must be read and written within microseconds; flash is non-volatile for recovery after power loss; only a short history is needed on the device.
Patient records (hot, current patients)	Database buffer pool in RAM, backed by NVMe SSD, replicated to the standby node	Frequent, random, small reads and writes needing sub-millisecond access and durability.
Patient records (old / history)	SSD or HDD tiers and network storage with backup	Rarely accessed; cost per TB matters; look-up in milliseconds is acceptable.
MRI images (being processed)	Main memory and NVMe scratch space of the imaging server	Large sequential data (hundreds of MB per study) processed by many cores.
MRI images (archive)	Network storage / HDD arrays (PACS) with off-site copy	Very large volume, rare access, must be retained for years.

9.3 Programmed I/O, interrupt-driven I/O and DMA

Method	How it works	CPU involvement	Suitable device in the hospital
Programmed I/O (polling)	CPU repeatedly reads the device status register and moves each word	100 % of the transfer time	Very simple devices, boot code
Interrupt-driven I/O	Device interrupts when ready; the CPU transfers one unit per interrupt	Only at each interrupt, but one interrupt per word or small unit	Monitor sensor events, keyboard, touch screen, alarm button

Method	How it works	CPU involvement	Suitable device in the hospital
DMA (direct memory access)	CPU programs the DMA controller with address and length; the controller moves the whole block between device and memory and interrupts once at the end	Setup plus one interrupt per block	MRI scanner, disk and network cards, bulk transfers

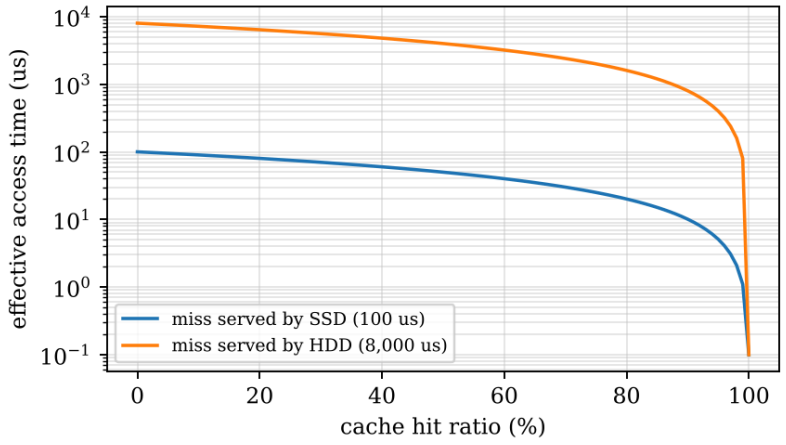
The MRI scanner produces hundreds of MB per study, so DMA is the right choice: the CPUs of the imaging server stay free to process earlier slices while the next ones arrive.

9.4 Effect of caching on the speed of record look-up

The effective access time is EAT = h × t_cache + (1 − h) × t_miss, where h is the hit ratio. Taking t_cache = 0.1 µs (RAM) and t_miss = 100 µs (SSD):

Hit ratio h	EAT with SSD miss (100 µs)	EAT with HDD miss (8,000 µs)
0 % (no cache)	100 µs	8,000 µs
50 %	50.05 µs	4,000 µs
95 %	5.1 µs	400 µs
99 %	1.1 µs	80 µs

Figure: Effect of caching on record look-up



Raising the hit ratio from 95 % to 99 % improves the average look-up by a factor of about 4.6 (SSD) to 5 (HDD). Record access is highly local (the patients currently admitted are read repeatedly), so a buffer pool large enough to hold the active records gives the very high hit ratio the cluster needs.

Alternative Concept and Complexity Analysis – interrupt-driven I/O (one interrupt per word) for the MRI scanner

Solution under the alternative. The scanner interrupts the CPU for every 4-byte word; the interrupt handler copies the word from the device register into memory. The image is 500 MB = 524,288,000 bytes.

Criterion	DMA, 64 KB blocks (original)	Interrupt per 4-byte word (alternative)
Number of interrupts	524,288,000 / 65,536 = 8,000	524,288,000 / 4 = 131,072,000
CPU time, assuming 5 µs per DMA completion and 2 µs per word interrupt (typical)	8,000 × 5 µs = 40 ms	131,072,000 × 2 µs = 262 s (4.4 minutes of one CPU)
Memory buffers	Pinned DMA buffers, e.g. two 64 KB buffers for double-buffering (128 KB), plus the 500 MB destination	Only the device register and the destination; no staging buffer, but the data passes through the CPU

Complexity. The number of interrupts is O(n / w) for words of w bytes and O(n / B) for DMA blocks of B bytes; with w = 4 and B = 65,536 the DMA method is 16,384 times fewer interrupts, and 6,550 times less CPU time in this example, because the cost per byte is not O(1) per byte any more but O(1) per block. The buffer space is O(w) = O(1) for the alternative against O(B) for DMA, which is a small price. Also, at 2 µs per interrupt the CPU would be saturated whenever the scanner delivers more than 2 MB/s (4 bytes per 2 µs), so the other jobs would stall. **Judgement:** DMA is retained for the MRI scanner; interrupt-driven I/O is kept for the low-rate monitor sensors.

Evidence

The data-placement plan (9.2) with typical access times (9.1), the I/O method for each device (9.3), the caching calculation (9.4) and the interrupt and CPU-time calculation for the 500 MB image.

Task 10 – Measure and Compare (4 Marks, K5)

10.1 Method

The benchmark bench.c (Task10 folder) creates a 64 MiB test file, then performs the operation named on the command line and reports the elapsed time with clock_gettime(CLOCK_MONOTONIC). The measured programs were also run under strace -c (system-call counts) and the bash time command (real, user and sys time). Environment: Ubuntu 24.04.5 LTS, gcc 13.3.0, kernel 6.18, 1 virtual CPU, file cache warm. Each measurement is a single run, except process/thread creation, which was repeated three times.

10.2 Number of system calls for different buffer sizes (strace -c and program count)

Method (64 MiB file)	read() calls returning data	Total read()/write() from strace -c	Other calls
read/write, b = 1	67,108,864	not traced (program count)	–
read/write, b = 512	131,072	program count	–
read/write, b = 4,096	16,384	16,386 read / 32,769 write	–
read/write, b = 65,536	1,024	program count	–
read/write, b = 1 MiB	64	program count	–
stdio fread/fwrite	16,384	16,386 read / 32,769 write	–
mmap	0	1 read / 16,385 write (these belong to start-up and to creating the test file)	10 mmap, 3 munmap, 1 ftruncate

The 32,769 writes are 16,384 for creating the test file plus 16,384 for the copy plus 1 for the printed result; the extra 2 reads are the end-of-file read and the loader’s read.

10.3 Time spent in user mode and kernel mode (the time command)

Method (64 MiB file)	real (s)	user (s)	sys (s)	Copy time (s)
read/write, b = 512	0.154	0.008	0.144	0.115

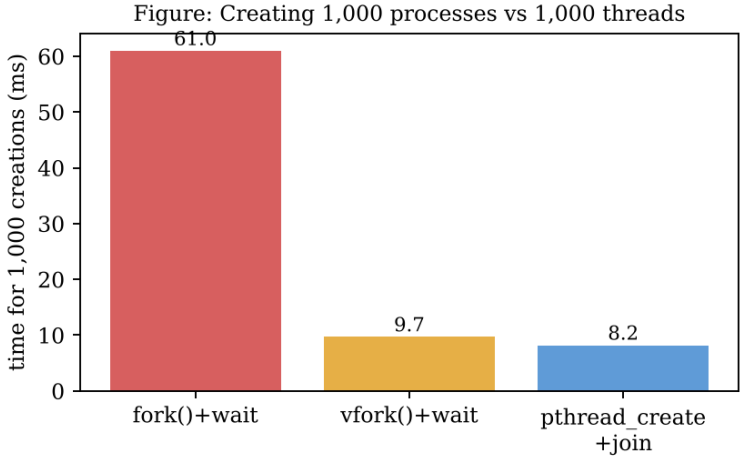
Method (64 MiB file)	real (s)	user (s)	sys (s)	Copy time (s)
read/write, b = 4,096	0.172	0.000	0.084	0.100
read/write, b = 65,536	0.184	0.004	0.073	0.093
stdio fread/fwrite	0.190	0.005	0.077	0.100
mmap	0.173	0.004	0.063	0.089
1,000 × fork + wait	0.058	0.024	0.028	0.057
1,000 × thread create + join	0.015	0.001	0.009	0.013

"real" includes creating the test file, so it is larger than the copy time. The important column is **sys**: for b = 512 the program spent 0.144 s in the kernel and only 0.008 s in user code, so almost all the extra cost of small buffers is system-call overhead. Increasing the buffer to 4 KB cut the kernel time by 42 % and 64 KiB by 49 %.

10.4 Time to create 1,000 processes with fork() and 1,000 threads

Run	fork() + wait (s)	vfork() + wait (s)	pthread_create + join (s)
1	0.0817	0.0101	0.0088
2	0.0520	0.0091	0.0078
3	0.0494	0.0100	0.0080
Mean	0.0610 (61 µs each)	0.0097 (9.7 µs each)	0.0082 (8.2 µs each)

Figure: Creating 1,000 processes vs 1,000 threads



Under strace -c -f the thread test made 9,031 system calls in total, about 9 per thread (stack allocation, clone, synchronisation and stack release), which is more than one might expect for a “lightweight” object but still far cheaper than copying an address space.

10.5 How the measurements support or contradict the earlier judgements

Earlier judgement	Measurement	Verdict
A system call is much more expensive than a function call (Task 6)	Average 0.23 µs per read/write; 1-byte buffer 333 times slower than 4 KB	**Supported**
Fewer calls give better performance; the cost is in the kernel (Task 6)	For b = 512 sys time 0.144 s against user time 0.008 s	**Supported**
Bigger buffers are always better; 64 KiB best (Task 6)	4 KiB to 1 MiB differ by under 10 %; in one run 64 KiB was slower than 4 KiB (0.143 s vs 0.092 s), in another it was faster (0.093 s vs 0.100 s)	**Partly contradicted**: the gain flattens after 4 KiB and run-to-run noise exceeds the difference. The 64 KiB recommendation stays because it needs 16 times fewer calls, but 4 KiB would also be acceptable on this hardware.
stdio buffering behaves like a 4 KB buffer (Task 5)	Identical 16,386 reads and 32,769 writes	**Supported**
mmap saves system calls and copying (Task 6)	10 mmap calls instead of 16,384 reads; time 10–25 % lower	**Supported**, but the time gain is much smaller than the call-count gain because page faults replace system calls
Threads are cheaper to create than processes (Task 7)	Threads 7 times faster than fork, close to vfork	**Supported**

Earlier judgement	Measurement	Verdict
Cost of fork is dominated by copying page tables (Task 7)	The test process is tiny, so 61 µs is mostly fixed kernel work; a multi-GB database process would cost far more	**Not tested at scale**: stated as a limitation

Alternative Concept and Complexity Analysis – threads instead of processes

Solution under the alternative. The 1,000 workers are created with pthread_create() instead of fork(). The measured result is in 10.4: 8.2 ms against 61.0 ms for 1,000 creations, 7.4 times faster, and close to vfork.

Criterion	1,000 processes	1,000 threads
Creation time (measured)	about 61 µs each	about 8.2 µs each
Memory used	A new page table and kernel structures per process; data pages shared copy-on-write until written; every write copies a 4 KB page	One new stack per thread (8 MB of virtual address space by default, from ulimit -s, of which only the touched pages use memory) plus a small kernel structure; code, heap and data are shared
What is shared	Nothing by default (open files are inherited)	Address space, heap, global data, open files; separate stack, registers and thread ID

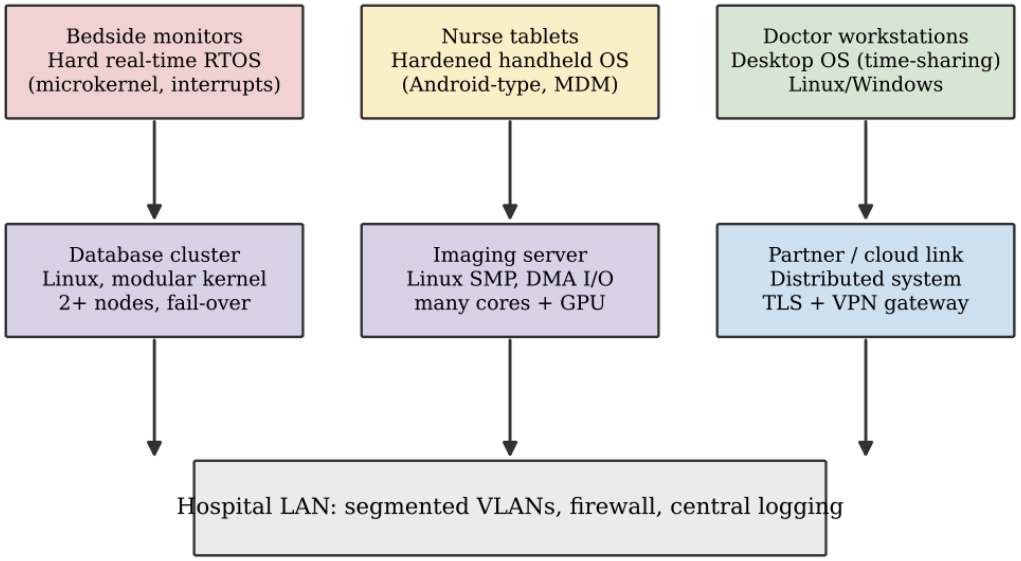
Explanation of the difference. fork() must duplicate the process descriptor, the table of open files, the signal handlers and the memory map, and set up copy-on-write page tables for every mapped page, work proportional to the size m of the parent's address space. A thread only needs a new stack and a new scheduling entity because everything else is shared. **Complexity:** creation time O(m) for processes and O(1) for threads; memory O(m) page-table entries plus O(w) copied pages for processes, O(1) per thread (stack size). The price of threads is no isolation: any thread can corrupt shared memory, so shared data needs locks.

Evidence

Tables 10.2 to 10.5 give the measured results, compared with expectations, with the differences explained. The raw files copy_times.txt, time_output.txt, proc_thread.txt and strace_*.txt are in the Task10 folder of the repository.

Task 11 – Recommend and Defend the Operating-System Plan (4 Marks, K5)

Figure: Recommended operating-system plan for the smart hospital



11.1 Recommended plan

Environment	Type and structure	Services (Task 4)	Protection	I/O method
E1 Bedside monitors	Hard real-time, microkernel-style RTOS (QNX-class)	Minimal: tasks, sensor I/O, secure link, watchdog	MPU task isolation, signed firmware, no user accounts	Interrupt-driven sensors
E2 Nurse tablets	Handheld OS, layered (Android-type), managed by MDM	Touch GUI, secure Wi-Fi, file store, power management	App sandbox, full-disk encryption, screen lock, remote wipe	Interrupt-driven; DMA for storage
E3 Doctor workstations	Desktop OS, monolithic/modular or hybrid	Full GUI, file system, printing, communication	User login with second factor, role-based access, patching	Interrupt-driven; DMA for disk and network

Environment	Type and structure	Services (Task 4)	Protection	I/O method
E4 Database	Clustered Linux, modular monolithic kernel, active/standby fail-over	Journaling file system, accounting, audit, replication	Authentication, ACL/SELinux, row-level security, encryption	DMA to NVMe/SAN
E5 Imaging server	SMP Linux with many cores and GPUs	Large-file I/O, batch queue, resource allocation	Isolated service account, segmented network, encrypted storage	DMA from scanner and storage
E6 Partner link	Distributed system, gateway on hardened Linux	TLS/VPN, authentication, audit log	Mutual authentication, per-hospital access control, encryption	DMA network cards

11.2 Evidence supporting the plan

- **Task 1 and Task 8:** only an RTOS with interrupts gives a bounded alarm delay (about 10 µs interrupt latency against 10 ms deadline; polling would need $T \leq 4$ ms and would never let the CPU sleep).
- **Task 3 and Task 10:** a system call costs about 0.23 µs on the test machine, so a microkernel's extra messages still use under 0.1 % of the deadline; the fault isolation is worth that cost.
- **Task 2:** the fail-over cluster gives about 32 seconds a year of ideal downtime against 8.8 hours for a single machine.
- **Task 6 and Task 10:** a 64 KiB buffer reduces calls 16-fold against 4 KiB and 65,000-fold against 1 byte; the 1-byte copy took 30.7 s against 0.09 s, so backup software must be written with large buffers.

- **Task 9:** DMA reduces the interrupt load for a 500 MB image from 131 million to 8,000.
- **Task 7 and Task 10:** process creation costs about 61 µs; with isolation required between commands, fork/exec is acceptable at the scale of a shell or service manager.

11.3 Limitations and risks

Risk	Effect	Mitigation
Several OS types increase administration effort	More images, tools and skills to maintain	Share one Linux base for E4, E5, E6 and workstations; central management for devices; automate patching
RTOS updates on monitors	A bad update can disable alarms	Staged roll-out on one ward, signed images, rollback partition, never update all monitors at once
Cluster split-brain or failed fail-over	Two active nodes or none	Quorum/fencing, tested fail-over drills, backups
Microkernel inter-process overhead	Slightly slower requests	Measured to be negligible against the deadline; test worst-case latency on target hardware
Lost or stolen tablets	Data exposure	Encryption, remote wipe, short screen lock
Our timings come from one virtual CPU	Absolute values will differ	Rely on counts and ratios; re-measure on target hardware before purchase

11.4 Suitability for the group of three hospitals

The plan scales because each monitor works independently (O(1) per device, with its alarm decided locally), the database cluster can add replicas or be partitioned by hospital, the imaging server is one per site, and the partner link generalises from one gateway to a mesh of three sites. Weak points are growth in the number of cross-site connections ($O(h^2)$ for h hospitals if every pair is linked directly; a central cloud or hub reduces this to O(h)) and

central administration, which should be strengthened with a unified identity system. The plan remains suitable; Task 12 extends it for home devices and the cloud.

Alternative Concept and Complexity Analysis – virtual machines on two large servers

Solution under the alternative. All server functions (database, imaging, workstation desktops through virtual desktops, partner gateway) would run as virtual machines on two large physical servers under a hypervisor; the monitors and tablets, being bedside and mobile devices, cannot move into a server, but their back-end services could.

Criterion	Separate physical machines	Virtual machines on two servers
Response time for the monitors	Alarms are decided on the monitor itself: 10 ms deadline is unaffected	Alarm path would go through virtual network and hypervisor scheduling, adding jitter of milliseconds when the host is busy; alarms must therefore stay on the device and only the central display/logging may be virtual
Memory and processor overhead	None	Hardware-assisted virtualisation typically costs a few percent of CPU (typical 2–10 %), plus hypervisor memory and one guest kernel per VM (hundreds of MB each)
Reliability	Failure affects one function; fail-over inside each cluster	Two hosts are two failure domains; if one host fails its VMs restart on the other in about a minute (if capacity was reserved); a hypervisor bug or overloaded host affects many functions at once

Complexity. With V virtual machines the memory overhead is O(V × K) where K is the guest-kernel and hypervisor memory per VM, and the CPU overhead is proportional to the

I/O and system-call rate (each privileged event can cause a VM exit). Hardware cost falls because idle capacity is shared (consolidation), but latency has a variance that grows with the host's load. **Judgement:** virtualise the non-critical services (workstation desktops, partner gateway, test systems) to save hardware, but keep the real-time monitors on dedicated RTOS hardware and keep the database and imaging on dedicated or CPU-pinned resources. The full move to two servers is rejected because it concentrates risk and cannot guarantee the monitors' 10 ms deadline.

Evidence

The complete plan table (11.1), the cross-references to measurements and calculations in Tasks 1–10 (11.2), the risk table (11.3) and the evaluation of the three-hospital group (11.4).

Task 12 – Day-4 Design Challenge: Re-evaluate and Adapt the Design (8 Marks, K5)

Changed requirement: the hospital adds remote monitoring devices in patients' homes that send readings over the internet, and moves patient records to a cloud service shared with the partner hospitals. **Decision: MODIFY** – the in-hospital core is retained, and the edges (home devices, records service, gateway, protection) are modified or added.

A. Impact Analysis (2 Marks)

Area	Affected?	How
OS type – monitors (E1)	Retained	The ward monitors and their 10 ms deadline are unchanged.
OS type – home devices (new, E7)	New	Battery- or mains-powered embedded devices outside the hospital's control: need a real-time embedded OS locally and a network connection that can fail.
OS type – database (E4)	Modified	Records move to a cloud distributed system; the on-site cluster becomes a primary or cache and a cloud replica appears.
OS type – partner link (E6)	Modified	Instead of a point-to-point link, partners use the shared cloud service with federated identity.
Services	Modified	Communication, accounting/audit and error detection grow in importance: remote update, store-and-forward, time synchronisation, audit of cross-organisation access.
Protection	Strongly affected	The trusted LAN boundary disappears: every device and user is on the internet, so mutual authentication, encryption in transit and at rest, secure boot, signed updates and strict per-tenant isolation are required.
I/O and memory	Slightly	More network I/O and bigger caches; DMA network cards remain.

B. Evaluation of the Original Design (2 Marks)

Criterion	Can the original design support the change?	Evidence
Response time	Home alarms: not if alarms depend on the internet. Internet round trips are typically 20–200 ms with no guarantee, and outages happen, so a remote 10 ms deadline cannot be guaranteed. Hospital monitors: still fine.	Task 8: bounded alarm path needs local interrupt handling; Task 3: cost of messaging is small but network latency is unbounded.
Reliability	Partly: the on-site cluster survives node failure but not loss of the internet link or of a cloud region; home devices need store-and-forward and a watchdog.	Task 2: fail-over design; availability formula $1 - (1 - A)^n$ requires independent failures.
Security	No: the design relied on a protected hospital network; no secure boot, device identity, remote-update signing or cross-organisation access control was planned.	Task 4 and Task 8: protection was designed around a local trusted environment.
Scalability	Partly: thousands of home devices each making many small connections stress the gateway; encryption adds CPU per request (many more system calls, Task 10).	Task 10: per-call cost is small but multiplies with the number of connections.

Conclusion: the original design cannot be **retained** unchanged because of security and the internet dependence; it does not need a complete **redesign** because the in-hospital requirements (bounded alarm delay, fail-over database, SMP imaging) are unchanged.

C. Adaptation or Redesign (2 Marks)

Component	Revised design
E7 Home monitoring device	Embedded hard/firm real-time OS (for example Zephyr, FreeRTOS with an MPU, or QNX) on a small board: local alarm (sound and indicator) decided on the device within the deadline; secure boot and signed over-the-air updates; TLS 1.3 with per-device certificates; local flash buffer for store-and-forward during outages; watchdog and low-power sleep between samples (interrupt-driven sensors).
Home gateway / phone app	Handheld OS (as E2), sandboxed app, encrypted storage, relays readings; the device can also connect directly over cellular.
Ingestion tier (in cloud or DMZ)	Linux servers in a distributed system behind load balancers; each connection is authenticated, rate-limited and logged; containers (namespaces and cgroups) isolate services and tenants.
Records service	Distributed database replicated across at least two cloud zones and the on-site cluster; hospital database cluster (E4) becomes primary or low-latency cache; consistency rules: patient writes are synchronous, history is replicated asynchronously.
Partner link (E6)	Federated identity and per-hospital authorisation on the shared service; audit trail of every access; data-protection compliance (for example India's DPDP Act 2023 and the hospital's consent rules) enforced by policy.
In-hospital core (E1–E5)	Unchanged, except that the monitors' central aggregator forwards alarms to the cloud asynchronously and never in the alarm's critical path.

D. Evidence-Based Defence (2 Marks)

Option	Advantage	Disadvantage	Verdict
Retain	No cost or risk of change	No device identity, secure update or internet security; alarm for home patients depends on the network	Rejected
Modify (chosen)	Keeps the proven hospital core; adds an RTOS for homes and a distributed records service with a stronger security model	More OS types and cloud dependence; need for operational maturity	**Accepted**
Redesign (all in cloud, thin devices)	Single platform, easiest central management	Monitors and home alarms would depend on the internet: no bounded 10 ms delay; outage stops care; migration risk	Rejected

Defence. (1) The 10 ms requirement is a property of the alarm path, and Tasks 1, 3 and 8 show that only a local real-time interrupt path can guarantee it; the internet cannot, so the alarm decision stays on the device, at home as in the ward. (2) Task 10 shows that system calls and process creation cost microseconds, so the extra encryption and logging for remote access fits inside the capacity of the ingestion servers; their main cost is scale (number of connections), solved by adding servers. (3) Task 2's availability model shows that independent replicas give very high availability, which the cloud replicas extend to a regional failure. (4) The modification brings the largest risk reduction (security) at the lowest cost because the stable and proven core is not rebuilt.

Trade-offs accepted: more types of OS to maintain (mitigated by automation and a common Linux base); reliance on a cloud provider (mitigated by on-site replica and contract terms); dependence on the internet only for non-critical reporting; some latency for cross-site record access (mitigated by caching with hit ratios as in Task 9, for example 99 % gives about 1 µs average lookup).

Evidence

The impact table (A), the evaluation table (B), the revised design (C) and the decision table with trade-offs (D), each referring back to the analyses and measurements of Tasks 1–11.

Submission Checklist and Viva Preparation

Submission checklist

Requirement	Status / where
GitHub repository CSA04-<RegisterNumber>-Assignment1 shared with the faculty	Create, then paste the link on the cover page and in this report
One folder per task with C sources, scripts, outputs and screenshots	Task01 … Task12 (see A.2)
Every Complete-the-Code program executed (Task 6, Task 7)	Task06/copy_syscalls.c and Task07/mini_shell.c with outputs
README.md: name, register number, gcc version, distribution, compile command, run steps, result note per task	Text prepared in A.1 and A.2
At least one commit per task (progressive history)	Commit messages listed in A.2
Report with tables, diagrams, strace and time outputs, measurements and complexity analyses for Tasks 1–12	This document

Viva preparation – questions you should be able to answer

1. What exactly happens between read() in your program and the data arriving in buf? (Task 5)
2. Why does a 1-byte buffer cost 333 times more than a 4 KB buffer in your measurement? (Task 6)
3. Why do you flush stdout before fork(), and why did the line appear twice with exit(3) and a redirect? (Task 7)
4. What is a zombie, and what happens to an orphan? (Task 7)
5. Why can a monitor not use polling or a general-purpose scheduler for its 10 ms alarm? (Tasks 1 and 8)

6. Why is DMA chosen for the MRI scanner, and how many interrupts would a 500 MB image need without it? (Task 9)
7. Why did you choose to modify, not redesign, the plan in Task 12? (Task 12)

Be ready to modify any program during the viva, for example to change the buffer sizes in Task 6, to run a different command in Task 7, or to count system calls with strace.